

The optimisers under the chain-ladder start: validation loss, reserve stability and training length

Three tables from `nncl_synthetic_layers_optimisers.csv` and what each of their columns means

Prepared for Francis

4 October 2026

Contents

1	What the tables show	2
2	The three tables	2
2.1	One hidden layer of 20 neurons	2
2.2	Three hidden layers of 20, 20 and 15 neurons	2
2.3	Three hidden layers of 20, 15 and 10 neurons	3
3	What each column means	4
3.1	The quantities behind the columns	4
3.2	Column 1: optimiser	4
3.3	Column 2: validation loss, mean	4
3.4	Column 3: sd	5
3.5	Columns 4 and 5: min and max	5
3.6	Column 6: below the start	5
3.7	Column 7: reserve sd (millions)	5
3.8	Column 8: steps used per run	6
4	Reading one row	6

1 What the tables show

Each table is for one architecture of the neural network chain ladder on the SynthETIC portfolio, fitted with the **chain-ladder start** (training `cl_start`: the output neuron is started in the chain-ladder factor of the training rows, and training is stopped early on the validation rows). A row is one optimiser. Every optimiser was fitted with 20 seeds, and a row summarises those 20 runs.

A **run** is a complete model: $J = 19$ networks, one per development period $j = 1, \dots, 19$, all with the same architecture, optimiser and seed. So a row of a table rests on $19 \times 20 = 380$ fitted networks.

The notation is that of the two companion explainers (*The mathematics behind the neural network chain ladder and its chain-ladder start* and *The fitting procedure of the neural network chain ladder*); the symbols needed here are repeated in Section 3. All amounts are in millions.

For scale: the validation loss of the chain-ladder start itself is 15,247.9 million (the same for every optimiser and seed), the true outstanding payments are 9,626.8 million, Mack's chain-ladder reserve is 9,479.4 million and Mack's standard error of that reserve is 285.4 million.

2 The three tables

2.1 One hidden layer of 20 neurons

optimiser	validation loss, mean	sd	min	max	below the start	reserve sd (millions)	steps used per run
SGD	15,223.0	2.9	15,217.7	15,227.4	0.16%	20.6	4,535
SGD momentum	15,217.2	3.6	15,212.7	15,227.4	0.20%	24.4	2,274
SGD Nesterov	15,216.8	3.9	15,212.0	15,227.4	0.20%	23.3	2,373
Adagrad	15,214.5	2.4	15,208.0	15,218.9	0.22%	72.6	2,192
Adadelat	15,213.9	1.1	15,211.7	15,215.7	0.22%	238.0	795
RMSprop	15,212.3	1.6	15,209.9	15,215.3	0.23%	137.7	808
Adam	15,213.0	1.9	15,208.1	15,215.9	0.23%	46.4	405
AdamW	15,213.0	1.9	15,208.1	15,215.9	0.23%	46.8	405
Adamax	15,213.4	1.7	15,209.9	15,215.7	0.23%	59.1	668
Nadam	15,213.0	2.0	15,207.9	15,215.9	0.23%	73.1	491

2.2 Three hidden layers of 20, 20 and 15 neurons

optimiser	validation loss, mean	sd	min	max	below the start	reserve sd (millions)	steps used per run
SGD	15,228.2	6.4	15,211.2	15,237.1	0.13%	23.0	4,475
SGD momentum	15,218.8	8.4	15,196.9	15,234.8	0.19%	29.6	2,640
SGD Nesterov	15,218.9	8.3	15,197.3	15,235.0	0.19%	29.3	2,722
Adagrad	15,214.6	5.9	15,197.6	15,224.8	0.22%	55.3	2,520
Adadelat	15,213.5	3.1	15,204.9	15,217.7	0.23%	250.6	773
RMSprop	15,209.0	4.1	15,201.7	15,215.5	0.26%	144.9	709
Adam	15,210.9	3.6	15,201.8	15,217.9	0.24%	55.3	377
AdamW	15,210.9	3.6	15,201.8	15,217.9	0.24%	55.4	376
Adamax	15,211.5	4.1	15,202.1	15,220.1	0.24%	84.7	560
Nadam	15,210.7	3.2	15,202.9	15,215.1	0.24%	107.3	420

2.3 Three hidden layers of 20, 15 and 10 neurons

optimiser	validation loss, mean	sd	min	max	below the start	reserve sd (millions)	steps used per run
SGD	15,234.6	5.3	15,225.7	15,243.4	0.09%	29.5	4,318
SGD momentum	15,223.6	6.5	15,213.0	15,241.8	0.16%	39.8	2,607
SGD Nesterov	15,223.7	6.5	15,213.2	15,241.8	0.16%	36.7	2,679
Adagrad	15,218.5	7.3	15,208.8	15,240.3	0.19%	38.8	2,757
Adadelat	15,214.2	2.8	15,207.9	15,218.9	0.22%	260.2	1,061
RMSprop	15,209.5	3.9	15,201.3	15,215.8	0.25%	151.5	923
Adam	15,211.2	4.6	15,204.6	15,222.3	0.24%	61.7	469
AdamW	15,211.2	4.6	15,204.6	15,222.3	0.24%	63.1	469
Adamax	15,212.3	5.3	15,205.2	15,223.4	0.23%	87.2	695
Nadam	15,210.5	3.8	15,205.1	15,217.4	0.25%	78.1	496

3 What each column means

3.1 The quantities behind the columns

For development period j , the learning cells $\mathcal{L}_j$ are the cells (accident year, feature value) with $C_{i,j-1}(\mathbf{x}) > 0$ and $C_{i,j}(\mathbf{x})$ observed. Row r of $\mathcal{L}_j$ has the volume $c_r = C_{i,j-1}(\mathbf{x})$, the response $d_r = C_{i,j}(\mathbf{x})$ and the feature $\mathbf{x}_r$. Keras cuts the rows, in their stored order, into the **training rows** $\mathcal{T}_j$ (the first 90%) and the **validation rows** $\mathcal{V}_j$ (the last 10%, which are the latest accident years of that development period). Only the training rows enter the gradient.

For a seed s , network j ends at parameters $\hat{\theta}_j^{(s)}$ after $t_j^{*(s)}$ gradient steps, and gives the factors $\hat{f}_r^{(s)} = f(\mathbf{x}_r; \hat{\theta}_j^{(s)})$. From these, three numbers are computed for the run with seed s :

$$L_{(s)}^{\mathcal{V}} = \frac{1}{u} \sum_{j=1}^J \sum_{r \in \mathcal{V}_j} \frac{(d_r - \hat{f}_r^{(s)} c_r)^2}{c_r}, \quad \hat{R}_{(s)} = \text{the reserve (5.1) of the run}, \quad t_{(s)}^* = \sum_{j=1}^J t_j^{*(s)}, \quad (1)$$

with $u = 10^6$. In the file `nncl_synthetic_layers_optimisers.csv` these are the columns `loss_vali`, `reserve` and `epochs_used` of the row of that run. Each table column is a summary of one of them over the $S = 20$ seeds $s = 2026, \dots, 2045$.

3.2 Column 1: optimiser

The rule that turns the gradient of the training loss into a parameter update. It is the only thing that differs between the rows of a table: the data, the split, the architecture, the starting factor, the seeds and the stopping rule are the same in every row.

Each optimiser runs at one learning rate, set in `config.yml` (`nncl: synthetic: optimizers`): 0.001 for SGD, SGD with momentum (momentum 0.9), Nesterov, RMSprop, Adam, AdamW, Adamax and Nadam; 0.01 for Adagrad; 1 for Adadelta. These are the Keras defaults except for the three SGD variants (default 0.01, at which they diverged in trial fits), Adagrad and Adadelta (default 0.001, at which they hardly move). The update rules are written out in Section 9 of the first companion.

3.3 Column 2: validation loss, mean

$$\bar{L}^{\mathcal{V}} = \frac{1}{S} \sum_s L_{(s)}^{\mathcal{V}}. \quad (2)$$

$L_{(s)}^{\mathcal{V}}$ in (1) is the weighted squared error of the fitted factors on the validation rows, added over the 19 development periods. Three things about it:

- **What is being measured.** For each validation cell the network predicts $\hat{f}_r c_r$ for the payment d_r . The squared error is divided by the volume c_r , because under the chain-ladder model the variance of d_r is proportional to c_r ; this is the same loss the network is trained on, evaluated on rows it was not trained on.
- **Why it is in millions.** A term $(d - \hat{f}c)^2/c$ is a payment squared over a payment, so the loss has the unit of the payments. It is not a percentage and has no meaning as an absolute number; it is only comparable with other losses on the same rows.
- **Why it is a sum over j .** The 19 networks are separate models with separate validation rows. Adding their losses gives one figure per run. The early development periods dominate it: the first period alone contributes about 12,170 of the roughly 15,210.

A lower value means the factors fit the held-out cells (the latest accident years of each period) better. Because the validation rows also choose the stopping step, the value is slightly optimistic as a forecast of the error on new data; between rows of a table that affects all optimisers alike.

3.4 Column 3: sd

$$\text{sd}(L^\mathcal{V}) = \sqrt{\frac{1}{S-1} \sum_s (L_{(s)}^\mathcal{V} - \bar{L}^\mathcal{V})^2}, \quad (3)$$

the sample standard deviation of the validation loss over the seeds. The seed fixes the random starting weights of the hidden layers and nothing else: the data and the split are the same for every seed. So this column measures how much the validation fit depends on the random start of the optimisation. It is not a standard error of the mean (that would be the sd divided by $\sqrt{S}$) and it says nothing about the randomness of the claims.

It is the yardstick for column 2: two optimisers whose means differ by less than about one sd cannot be told apart by their validation loss.

3.5 Columns 4 and 5: min and max

$$\min_s L_{(s)}^\mathcal{V} \quad \text{and} \quad \max_s L_{(s)}^\mathcal{V}, \quad (4)$$

the best and the worst validation loss among the 20 seeds. Together they give the range, which shows whether the sd is driven by one unusual seed. By the stopping rule (Column 6) no value can exceed the loss of the chain-ladder start, 15,247.9, beyond rounding.

3.6 Column 6: below the start

$$1 - \frac{\bar{L}^\mathcal{V}}{L_0^\mathcal{V}}, \quad L_0^\mathcal{V} = \frac{1}{u} \sum_{j=1}^J \sum_{r \in \mathcal{V}_j} \frac{(d_r - \hat{f}_{j-1}^\mathcal{T} c_r)^2}{c_r} = 15,247.9, \quad \hat{f}_{j-1}^\mathcal{T} = \frac{\sum_{r \in \mathcal{T}_j} d_r}{\sum_{r \in \mathcal{T}_j} c_r}. \quad (5)$$

$L_0^\mathcal{V}$ is the validation loss of the networks *before any training*: every network starts as the plain chain-ladder factor $\hat{f}_{j-1}^\mathcal{T}$ of its training rows, the same factor for every feature value. It does not depend on the optimiser or the seed.

The column is the relative reduction of the validation loss that training achieves on average: how much better than the plain chain ladder the covariate-dependent factors predict the held-out cells. It cannot be negative, because a network that never beats its start on the validation rows is put back to the start. Values around 0.2% say that the networks improve on the chain ladder, but by very little.

3.7 Column 7: reserve sd (millions)

$$\text{sd}(\hat{R}) = \sqrt{\frac{1}{S-1} \sum_s (\hat{R}_{(s)} - \bar{R})^2}, \quad \bar{R} = \frac{1}{S} \sum_s \hat{R}_{(s)}, \quad (6)$$

the standard deviation over the seeds of the total reserve. The reserve of a run is the sum over the diagonal cells of the latest payment times the product of the fitted factors of the remaining development periods, less the latest payment, plus the zero claims reserve.

This is the only column that is about the reserve, the quantity the model exists for. It answers: if I fit the same model again with another seed, by how much does my reserve change? It is a measure of

the stability of the optimiser, not of its accuracy: it does not involve the true outstanding payments, and a model can be stable and biased.

Two reference points: Mack’s standard error of the chain-ladder reserve is 285.4 million, so a reserve sd of 20 to 60 is small against the uncertainty of the reserve itself, while 140 to 260 (RMSprop, Adadelta) is of the same order; and 1% of the true reserve is 96 million.

The column also shows that a small spread of the validation loss does not imply a small spread of the reserve. The reserve multiplies up to 19 factors per cell and is dominated by the cells with the largest volumes, so small differences in the factors of single cells, which hardly move the summed loss, can move the reserve.

3.8 Column 8: steps used per run

$$\bar{t}^* = \frac{1}{S} \sum_s t_{(s)}^* = \frac{1}{S} \sum_s \sum_{j=1}^J t_j^{*(s)}. \quad (7)$$

$t_j^{*(s)}$ is the number of gradient steps after which network j had its smallest validation loss; the weights of that step are the ones kept. It is 0 if no step improved on the start (the network stays the chain-ladder factor) and at most 500, the cap on the number of epochs. With a batch that holds all training rows one epoch is one step, so “epochs” and “steps” are the same here. The column adds the 19 networks of a run and averages over the seeds, so its maximum is $19 \times 500 = 9,500$.

It measures how far each optimiser travels before early stopping ends it, which is its speed in the sense of training effort. It is not the number of steps actually computed: training runs 20 further steps after the best one before it stops (the patience), so the computed steps are larger (the column `epochs_run` of the file). A large value combined with a high validation loss, as for SGD, indicates an optimiser that is still improving slowly when it reaches the cap.

4 Reading one row

Take Adam with one hidden layer of 20 in the first table. Over the 20 seeds the 19 networks have a validation loss of 15,213.0 million on average (column 2), with a standard deviation of 1.9 across the seeds (column 3) and values between 15,208.1 and 15,215.9 (columns 4 and 5). The chain-ladder start has 15,247.9, so training reduces the validation loss by 0.23% (column 6). The reserves of the 20 runs have a standard deviation of 46.4 million (column 7), about a sixth of Mack’s standard error. A run uses 405 gradient steps in total over its 19 networks (column 8), about 21 per network, far below the cap of 500.